IA JOGANDO DUCK HUNT

Da tela ao tiro: visão computacional com YOLOv5 ligada ao código.

MAPA DE ESTUDO

A4 • HORIZONTAL

1 O PROBLEMA

Objetivo: acertar os patos olhando só a tela.

Agente: perceber → decidir → agir.

Detecção: diz **o que** há e **onde** está (caixa + classe + confiança).

Sem treino: YOLOv5n pronto, treinado em fotos (COCO, 80 classes).

O COCO não tem pato.
O modelo vê o pato como “pipa” (kite).

2 CAPTURAR A TELA

A cada 200 ms: stage → canvas → ImageBitmap.

Web Worker: a IA roda em outra thread; o jogo não trava.

postMessage(msg, [bitmap]): transfere a imagem sem copiar.

Resposta: { type: 'prediction', x, y, score }.

O céu sai transparente → preto.
A cor de fundo não chega ao modelo.

3 PREPARAR O TENSOR

fromPixels: imagem → [600, 800, 3].

resizeBilinear: → [640, 640, 3].

div(255): pixels de 0 a 1. **expandDims(0):** → **[1, 640, 640, 3]** (lote).

tf.tidy: libera tensores da memória da GPU.

Esticar 4:3 para 1:1 deforma o pato.
Coordenadas normalizadas desfazem isso.

4 DETECTAR (YOLOv5n)

Uma passada pela rede → todas as caixas.

1,88 M parâmetros; graph model (só inferência).

Saídas: boxes [1, 100, 4], scores, classes.

NMS: apaga caixas repetidas (já dentro do modelo).

Warmup no início. **executeAsync** por causa do NMS.

Score ≠ certeza.
É a confiança do modelo, não uma chance validada.

5 FILTRAR E MIRAR

Filtro: score ≥ 0,4 **e** classe 'kite'.

Caixa 0 a 1 × largura/altura → pixels.

Centro: x1 + largura/2; y1 + altura/2.

Generator: um yield por pato → uma mensagem → **um tiro**.

Pato visto como “person” é descartado.
No céu azul isso acontece muito.

6 ATIRAR E AVALIAR

aim.setPosition → **handleClick**: o mesmo clique do mouse.

Acerto: pato a menos de **60 px** do tiro.

Precisão: tiros certos. **Recall:** patos achados.

Teste: precisão **100%**; recall **46%** por frame.

Só 3 balas por onda: errar custa mais que esperar.

Sem backpressure: se a IA levar > 200 ms,
os frames se acumulam e a mira atrasa.

SIGA AS SETAS: 1 → 2 → 3 • depois 4 → 5 → 6, da direita para a esquerda.

Síntese do código analisado (machine-learning/main.js e worker.js). Teste: 40 cenas com os sprites do jogo, mesmo modelo e pré-processamento. • Imprimir em A4, orientação paisagem.